

Nama : Radhit Akriandra
NRP : 5025251084

Soal 1: LOMotion

Penjelasan

- Jika arah Iel belum lurus ke koordinat Dan, maka Iel akan menyatakan "Aku akan berputar ke arah tujuan!". Rumus yang digunakan Iel untuk menghitung arah tujuan adalah:

$$\theta_2 = \arctan\left(\frac{y_2 - y_1}{x_2 - x_1}\right)$$

Pada kode, gunakan `atan2(y,x)` dari library `<math.h>`.

Iel akan menganggap arahnya sudah lurus dengan Dan jika perbedaan arahnya sekarang dan arah tujuan dia kurang dari 0,01.

$$|\theta_1 - \theta_2| < 10^{-2}$$

- Jika arah Iel sudah lurus dengan koordinat Dan, maka Iel akan menyatakan "Aku akan berjalan lurus ke Dan!"
- Jika koordinat Iel sama dengan koordinat Dan, maka Iel akan menyatakan "Halo, Dan, aku Iel."

Solusi

Kita dapat mengecek apakah $(x_1 == x_2) \ \&\& \ (y_1 == y_2)$. Jika kondisi tersebut terpenuhi, maka koordinat Iel dan koordinat Dan sudah sama.

Untuk kondisi lain, kita dapat mengimplementasikan rumus untuk menghitung arah tujuan terhadap koordinat Dan. Jika perbedaan sudut yang dibentuk antara arah Iel sekarang dengan arah koordinat Dan kurang dari 0.01, maka arah Iel dengan Dan sudah lurus.

Berikut kode untuk soal di atas.

Locomotion.c

```
#include <stdio.h>
#include <math.h>
#define pi M_PI
int main(){
    double x1, y1, x2, y2, theta1;
    scanf("%lf %lf %lf %lf %lf", &x1, &y1, &theta1, &x2, &y2);
    double theta2 = atan2(y2 - y1, x2 - x1);
    if(x1 == x2 && y1 == y2){
        printf("Halo, Dan, aku Iel.");
        return 0;
    }
    double arah = fabs(theta1 - theta2);
    if(arah < 0.01){
        printf("Aku akan berjalan lurus ke Dan!");
    }else{
        printf("Aku akan berputar ke arah tujuan!");
    }
    return 0;
}
```

Btw soal ini kemaren saya salah ketik. Seharusnya Iel, saya malah ketik Lel :))))))

Soal 2: Satu, Dua, Tiga

Penjelasan

Diberikan 3 ranting dengan panjang masing-masing a , b , dan c . Apakah memungkinkan untuk membentuk segitiga siku-siku dari ketiga ranting tersebut?

- Jika memungkinkan, cetak "Yatta, aku bisa membentuk segitiga!"
- Jika tidak, cetak "Gomenasai, segitiga tidak dapat dibuat."

Solusi

Pertama-tama, urutkan terlebih dahulu panjang ranting dari yang terkecil ke yang terbesar sehingga didapat kemungkinan ranting terpanjang adalah ranting dengan panjang c .

Dengan Teorema Pythagoras, kita tahu bahwa untuk semua segitiga siku-siku berlaku

$$x^2 + y^2 = z^2$$

dengan z adalah hipotenusa, dan x, y sisi yang tersisa. Agar sebuah segitiga siku-siku dapat dibentuk, maka kita dapat mengecek apakah $a^2 + b^2 \leq c^2$. Jika $a^2 + b^2$ melebihi c^2 , maka ranting dengan panjang c tidak dapat menjadi hipotenusa segitiga.

Berikut kode untuk soal di atas.

Satuduatiga.c

```
#include <stdio.h>
int main(){
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);
    if(a > b){
        int temp = a;
        a = b;
        b = temp;
    }
    if(a > c){
        int temp = a;
        a = c;
        c = temp;
    }
    if(b > c){
        int temp = b;
        b = c;
        c = temp;
    }
    if((long long)a*a + (long long)b*b <= (long long)c*c){
        printf("Yatta, aku bisa membentuk segitiga!");
    }else{
        printf("Gomenasai, segitiga tidak dapat dibuat.");
    }
    return 0;
}
```

Catatan: Karena a, b, c bisa sampai 10^{10} kita lakukan type casting ke tipe data long long.

Soal 3: The Perfect Pair

Penjelasan

Diberikan dua bilangan E dan S. Bilangan E dan S dikatakan **perfect pair** jika **tidak ada bit yang bernilai 1 yang bertabrakan**.

- Jika E dan S adalah perfect pair, keluarkan "hop on roblox!"
- Jika bukan, keluarkan "we'll try again tomorrow :("

Solusi

Untuk mengecek apakah dua bilangan **perfect pair** atau tidak. Kita dapat menggunakan operasi bitwise & (AND). Operasi & akan menghasilkan 1 jika kedua bit pada posisi sama bernilai 1. Jika salah satu atau keduanya bernilai 0, maka hasilnya 0.

Dengan demikian, dua bilangan E dan S dikatakan **perfect pair** jika hasil operasi $E \& S = 0$. Artinya tidak ada bit yang saling bertabrakan pada representasi biner antara dua bilangan tersebut.

Berikut kode untuk soal di atas.

Perfectpair.c

```
#include <stdio.h>

int main(){
    unsigned long long e, s;
    scanf("%llu %llu", &e, &s);
    if((e & s) == 0){
        printf("hop on roblox!");
    }else{
        printf("we'll try again tomorrow :(");
    }
}
```

Soal 4: Banyak Banget Calculation

Penjelasan

Buatlah program yang dapat menghitung total biaya parkir sesuai dengan jenis kendaraan, lama parkir, dan opsi pembayaran yang dipilih dengan ketentuan berikut:

1. Mobil

- 3 jam pertama: Rp10.000
- Setiap jam berikutnya: +Rp2.000
- Setiap 24 jam: +Rp150.000

2. Sepeda Motor

- Sebelum 24 jam: Rp3.000
- Setiap 24 jam: +Rp50.000

Opsi Diskon:

- Opsi a: Tanpa potongan
- Opsi b: Potongan 20
- Opsi c: Potongan 10

Solusi

Pertama tama, kita akan melakukan input dan mengecek apakah input waktu masih dalam kondisi yang wajar.

Calculation.c

```
#include <stdio.h>
int main(){
    int dd1, hh1, mm1, dd2, hh2, mm2;
    char chr1, chr2, jenis, bayar;
    scanf("%d %d%c%d %d %d%c%d %c %c", &dd1, &hh1, &chr1, &mm1, &dd2, &hh2, &chr2,
    &mm2, &jenis, &bayar);
    if(dd2 < dd1 || (dd2 == dd1 && hh2 < hh1) || (dd2 == dd1 && hh2 == hh1 && mm2 <
    mm1)){
        printf("Tidak mungkin");
        return 0;
    }
}
```

Di sini kita akan menghitung durasi parkir dan normalisasi waktu dengan cara sistem pinjam-meminjam, seperti melakukan penambahan jam atau penambahan hari. Setelah normalisasi waktu, kita simpan masing-masing dalam variabel `display_hari`, `display_jam`, dan `display_menit`.

```
int jumlah_hari = dd2 - dd1;
int jumlah_jam = hh2 - hh1;
int jumlah_menit = mm2 - mm1;

if(jumlah_menit < 0){
    jumlah_jam -= 1;
    jumlah_menit += 60;
}

if(jumlah_jam < 0){
    jumlah_hari -= 1;
    jumlah_jam += 24;
}

int display_hari = jumlah_hari;
int display_jam = jumlah_jam;
int display_menit = jumlah_menit;

int billing_jam = jumlah_jam;
if(jumlah_menit > 0){
    billing_jam += 1;
}
```

Selanjutnya, kita tinggal implementasikan syarat yang diminta soal dan kita simpan biaya total dalam variabel `ans`.

Mobil

- 3 jam pertama: Rp10.000
- Setiap jam berikutnya: +Rp2.000
- Setiap 24 jam: +Rp150.000

Sepeda Motor

- Sebelum 24 jam: Rp3.000
- Setiap 24 jam: +Rp50.000

```
int ans = 0;

if(jenis == 'M'){
    if(jumlah_hari == 0){
        if(billing_jam <= 3){
            ans += 10000;
        } else {
            ans += 10000 + 2000 * (billing_jam - 3);
        }
    }else{
        ans += 10000 + jumlah_hari * 150000 + 2000 * billing_jam;
    }
}else if(jenis == 'S'){
    if(jumlah_hari == 0){
        if(billing_jam > 0){
            ans += 3000;
        }
    }else{
        ans += jumlah_hari * 50000;
        if(billing_jam > 0){
            ans += 3000;
        }
    }
}
}
```

Selanjutnya, terdapat input berupa jenis diskon yang dipilih. Terdapat 3 pilihan diskon, yaitu:

- Opsi a: Tanpa potongan
- Opsi b: Potongan 20%
- Opsi c: Potongan 10%

Kita tinggal cek jenis diskon yang diminta apakah opsi (b) atau (c). Jika pilihan diskon adalah opsi (b), maka diskon bernilai $20\% \times \text{biaya akhir}$. Sehingga biaya akhirnya dikurangi dengan diskon. Demikian juga terhadap opsi (c).

```
if(bayar == 'b'){
    int diskon = ans * 20 / 100;
    if(diskon > 10000){
        diskon = 10000;
    }
    ans -= diskon;
}else if(bayar == 'c'){
    int diskon = ans * 10 / 100;
    if(diskon > 20000){
        diskon = 20000;
    }
}
```

```
    }  
    ans -= diskon;  
}  
printf("Anda telah berada di sini selama %d hari, %d jam, dan %d menit. Anda harus  
membayar sebesar %d rupiah", display_hari, display_jam, display_minut, ans);  
}
```

Mungkin soal ini bisa dikerjakan dengan cara mengonversi waktunya menjadi total menit, tetapi saya malas ngoding dari awal :)))